Laboratorium 1

Analiza błędów

Autorzy:

Kacper Hawro, Jan Ślosarczyk

01.04.2026

---

1. Zadanie 1

1.1. Treść zadania

Oblicz przybliżoną wartość pochodnej funkcji, używając wzoru:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{1}$$

Sprawdź działanie programu dla funkcji $\tan(x)$ oraz $x = 1$. Wyznacz błąd, porównując otrzymaną wartość numerycznej pochodnej z prawdziwą wartością. Pomocna będzie tożsamość $\tan'(x) = 1 + \tan^2(x)$.

Na wspólnym rysunku przedstaw wykresy wartości bezwzględnej błędu metody, błędu numerycznego oraz błędu obliczeniowego $E(h)$ w zależności od h dla $h = 10^{-k}$, $k = 0, ..., 16$. Użyj skali logarytmicznej na obu osiach.

Wypisz wyznaczoną empirycznie wartość $h_{\min}$ oraz wartość $h_{\min}$ otrzymaną ze wzoru:

$$h_{\min} \approx 2\sqrt{\varepsilon_{\text{mach}}/M}, \quad M \approx |f''(x)| \tag{2}$$

Powtórz ćwiczenie używając wzoru różnic centralnych:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \tag{3}$$

Wypisz wartość $h_{\min}$ ze wzoru:

$$h_{\min} \approx \sqrt[3]{3\varepsilon_{\text{mach}}/M}, \quad M \approx |f'''(x)| \tag{4}$$

Porównaj wyznaczoną empirycznie wartość błędu obliczeniowego E(hmin), z jakim wyznaczone zostało $f'(x)$, przy zastosowaniu wzorów (1) oraz (3). Która metoda jest dokładniejsza?

1.2. Rozwiązanie

1.2.1. Różnica w przód

Pochodna funkcji $f(x) = \tan(x)$ w punkcie $x = 1$ określana jest wzorem:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{5}$$

Prawdziwa wartość pochodnej wyznaczana jest analitycznie ze wzoru $f'(x) = 1 + \tan^2(x)$.

Całkowity błąd metody $E(h)$ składa się z dwóch składowych:

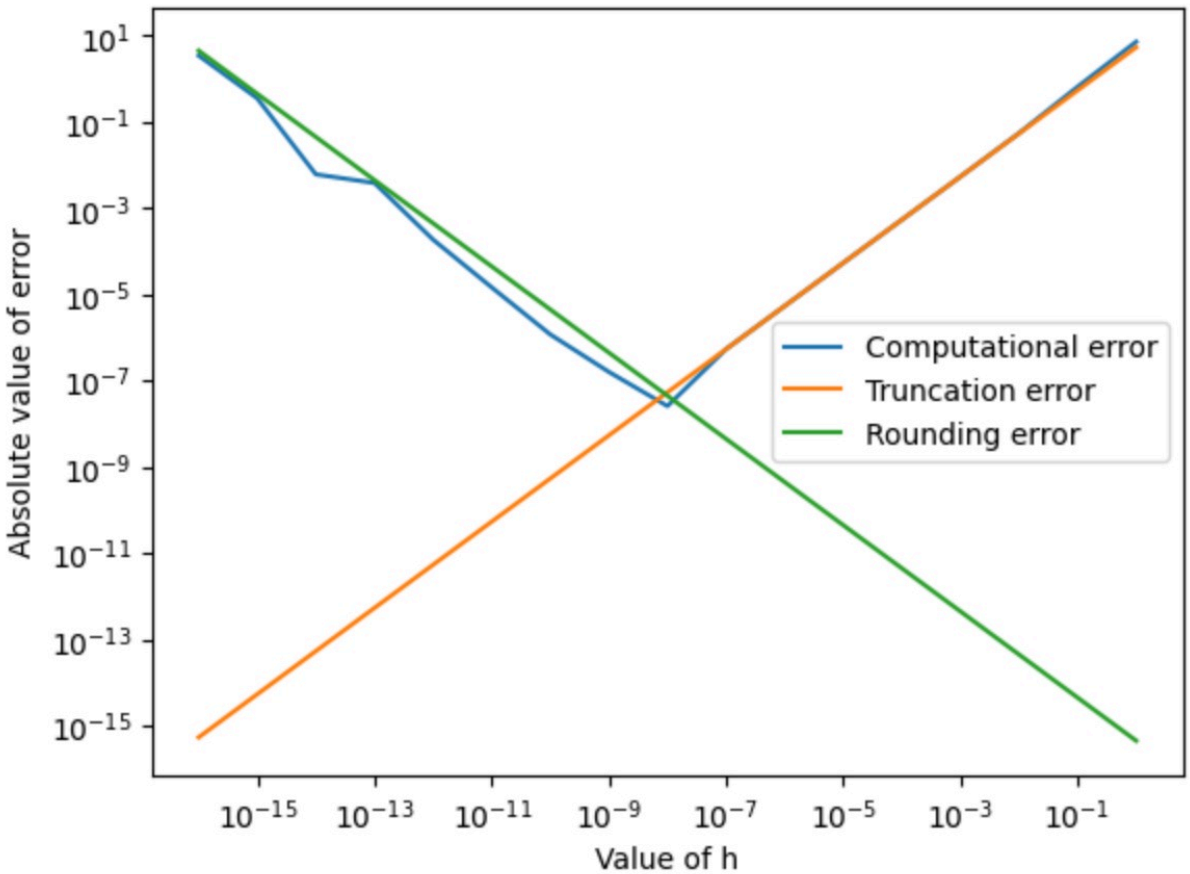
Błąd metody (obcięcia) - wynika z pominięcia wyrazów wyższego rzędu w rozwinięciu Taylora. Wynosi $\frac{M_2 \cdot h}{2}$, gdzie $M_2 \approx |f''(x)| = |2\tan(x)(1 + \tan^2(x))|$. Maleje liniowo wraz ze zmniejszaniem h.

Błąd numeryczny (zaokrąglenia) - rośnie przy zmniejszaniu h, ponieważ dla bardzo małych kroków $f(x+h) \approx f(x)$ i dochodzi do katastrofalnego odejmowania bliskich sobie wartości. Szacowany jest wzorem $\frac{2\varepsilon_{\text{mach}}}{h}$.

Oba błędy wyznaczają punkt optymalny h_{min}, w którym błąd obliczeniowy jest najmniejszy. Wartość ta wyznaczana jest ze wzoru:

$$h_{\text{min}} \approx 2\sqrt{\frac{\varepsilon_{\text{mach}}}{M_2}} \tag{6}$$

1.3. Wykresy i wyniki



The point at which the computational error has the smallest value:

```
h_min = 2 * np.sqrt(epsilon_mach/M_2)
print(h_min)
```

```
9.123695225180451e-09
```

The absolute value of the relative error at the designated point (x = 1)

```
h_value = np.array([h_min])
print(np.abs((forward_diff(f_tan, x, h_value) - d_f_tan(x)) / d_f_tan(x) ))
```

```
[5.33866349e-09]
```

1.3.1. Różnica centralna

Pochodna funkcji $f(x) = \tan(x)$ w punkcie $x = 1$ aproksymowana jest symetrycznym wzorem różnicowym:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \tag{7}$$

Dzięki symetrii wzoru wyrazy nieparzyste rozwinięcia Taylora znoszą się wzajemnie, co prowadzi do istotnie lepszych właściwości numerycznych w porównaniu z różnicą w przód.

Całkowity błąd metody $E(h)$ składa się z dwóch składowych:

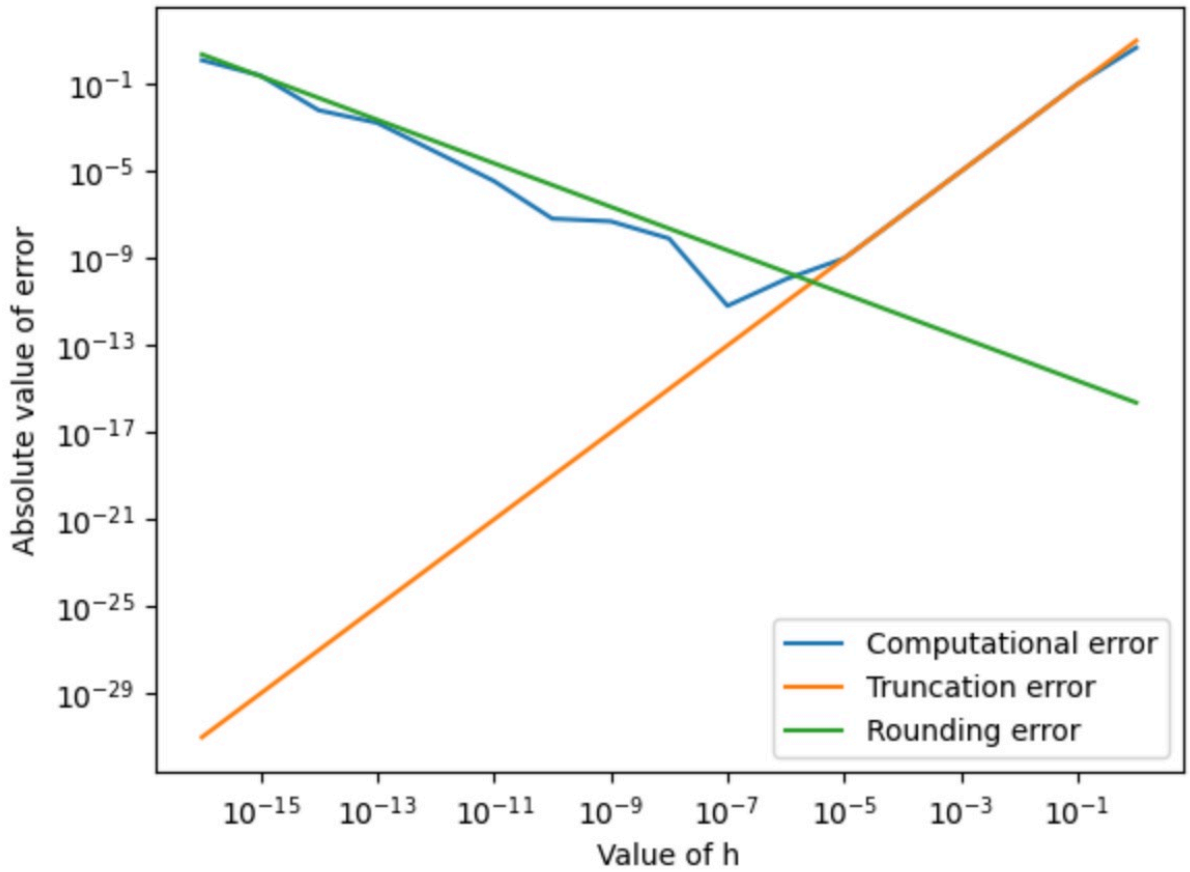
Błąd metody (obcięcia) - wynosi $\frac{M_3 \cdot h^2}{6}$, gdzie $M_3 \approx |f'''(x)| = |2(f'(x))^2 + 4\tan^2(x) \cdot f'(x)|$. Maleje proporcjonalnie do h^2, a nie liniowo jak w przypadku różnicy w przód — jest to zasadnicza przewaga tej metody.

Błąd numeryczny (zaokrąglenia) - szacowany wzorem $\frac{\varepsilon_{\text{mach}}}{h}$, rośnie przy zbyt małych wartościach h.

Ponieważ błąd metody maleje szybciej ($\sim h^2$), optymalny krok $h_{\min}$ jest większy, co dodatkowo ogranicza wpływ błędów zaokrąglenia. Wartość $h_{\min}$ wyznaczana jest ze wzoru:

$$h_{\min} \approx \sqrt[3]{\frac{3\varepsilon_{\text{mach}}}{M_3}} \tag{8}$$

1.4. Wykresy i wyniki



The point at which the computational error has the smallest value:

```
h_min = np.cbrt(3 * epsilon_mach / M_3)
print(h_min)
```

```
2.2732741568390613e-06
```

The absolute value of the relative error at the designated point (x = 1)

```
h_value = np.array([h_min])
print(np.abs((central_diff(f_tan, x, h_value) - d_f_tan(x)) / d_f_tan(x) ))
```

```
[2.53349201e-11]
```

Computational error

```
t_error_min_3 =  M_3 * ( h_min ** 2 )/6
r_error_min_3 = epsilon_mach / h_min_3

E_h_min_3 = t_error_min_3 + r_error_min_3
print(E_h_min_3)
```

```
1.4651418368766806e-10
```

1.5. Wnioski

Z obliczeń wynika, że metoda różnic centralnych jest o wiele dokładniejsza niż różnicy w przód, ponieważ błąd metody maleje proporcjonalnie do kwadratu, a nie liniowo. Pozwala uzyskać dużo mniejszy błąd (rzędu 10^{-11} zamiast 10^{-9}), a do tego optymalny krok $h_{\min}$ nie musi być aż tak mały, co wpływa na zmniejszenie się błędów zaokrągleń pochodzących od precyzji maszynowej.

2. Zadanie 2

2.1. Treść zadania

Przepisz poniższe wyrażenia tak, aby zmniejszyć błąd numeryczny:

(a) $\sqrt{x+1}-1, \quad x \approx 0$

(b) $x^2 - y^2, \quad x \approx y$

(c) $\frac{1-\cos x}{\sin x}, \quad x \approx 0$

(d) $\sin x - \sin y, \quad x \approx y$

(e) $\frac{a+b}{2}$ (bisekcja odcinka $[a, b]$)

(f) $\mathrm{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$ dla $\boldsymbol{x} = [x_1, ..., x_n]$

Dla dwóch wybranych podpunktów podaj przykładowe wartości argumentów oraz precyzję, przy której oryginalna i stabilna wersja różnią się wynikiem (przydatna biblioteka `decimal`).

2.2. Rozwiązanie

2.2.1. (a) $\sqrt{x+1}-1$

Stabilna wersja: $\sqrt{x+1}-1 = \frac{(\sqrt{x+1}-1)(\sqrt{x+1}+1)}{\sqrt{x+1}+1} = \frac{x}{\sqrt{x+1}+1}$

2.2.2. (b) $x^2 - y^2$

Stabilna wersja: $x^2 - y^2 = (x-y)(x+y)$

2.2.3. (c) $\frac{1-\cos x}{\sin x}$

Stabilna wersja: $\frac{1-\cos(x)}{\sin(x)} = \frac{(1-\cos(x))(1+\cos x)}{\sin(x)(1+\cos(x))} = \frac{1-\cos^2 x}{\sin(x)(1+\cos(x))} = \frac{\sin^2(x)}{\sin(x)(1+\cos(x))} = \frac{\sin(x)}{1+\cos(x)}$

2.2.4. (d) $\sin x - \sin y$

Stabilna wersja: $\sin x - \sin y = 2\sin\left(\frac{x-y}{2}\right)\cos\left(\frac{x+y}{2}\right)$

2.2.5. (e) $\frac{a+b}{2}$

Stabilna wersja: $\frac{a+b}{2} \Rightarrow a + \frac{b-a}{2}$

2.2.6. (f) $\text{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$ **dla** $\boldsymbol{x} = [x_1, ..., x_n]$

Stabilna wersja: $\text{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)} = \frac{\exp(x_i)\exp(-M)}{\sum_j \exp(x_j)\exp(-M)} = \frac{\exp(x_i - M)}{\sum_j \exp(x_j - M)}$,

dla $M = \max([x_1, ..., x_n])$

2.3. Przykłady liczbowe (dwa wybrane podpunkty)

Setting decimal precision:

```
getcontext().prec = 16
```

```
x = Decimal("124819e-14")
```

Equations:

```
original = (x+1).sqrt() - 1
stable = x / ((x+1).sqrt() + 1)
```

Answers:

```
print("Original equation: ", original)
print("Stable equation:   ", stable)
```

```
Original equation:  6.24095E-10
Stable equation:    6.240949998052527E-10
```

Rysunek 1: Podpunkt A

Setting decimal precision:

```
getcontext().prec = 19
```

```
x = Decimal("3333333333.333333333")
y = Decimal("3333333333.333333332")
```

Equations:

```
original = x**2 - y**2
stable = (x-y)*(x+y)
```

Answers:

```
print("Original equation: ", original)
print("Stable equation:   ", stable)
```

```
Original equation:  1E+1
Stable equation:    6.666666666666666665
```

Rysunek 2: Podpunkt B

2.4. Wnioski

Niektóre funkcje można przekształcić algebraicznie i dzięki temu otrzymać większą precyzję cyfr znaczących.

3. Zadanie 3

3.1. Treść zadania

Rozważamy funkcję:

$$f(x) = \begin{cases} 1 & \text{dla } x = 0 \\ \frac{e^x - 1}{x} & \text{dla } x \neq 0 \end{cases} \tag{9}$$

1. Wykaż, że $\lim_{x \to 0} f(x) = 1$.
2. Narysuj wykres $f(x)$ dla $x = 10^{-k}$, $k = 0, ..., 16$.
3. Dodaj wykres używając matematycznie równoważnej formuły opartej na $y = e^x$; $f(x) = \frac{y-1}{\log(y)}$.
4. Zaproponuj sposób obliczania $f(x)$ o lepszych własnościach numerycznych (rozwinięcie Taylora). Dodaj wykres.
5. Dodaj wykres korzystając z funkcji `expm1`. Skomentuj wyniki.

3.2. Rozwiązanie

3.2.1. Dowód granicy

Zgodnie z treścią zadania, rozważamy funkcję określoną wzorem:

$$f(x) = \begin{cases} 1 & \text{dla } x = 0 \\ \frac{e^x - 1}{x} & \text{dla } x \neq 0 \end{cases} \tag{10}$$

Naszym celem jest wykazanie, że $\lim_{\{x \to 0\}} f(x) = 1$. Ponieważ interesuje nas otoczenie punktu $x = 0$ (gdzie $x \neq 0$), analizujemy ułamek:

$$\lim_{x \to 0} \frac{e^x - 1}{x} \tag{11}$$

Przy bezpośrednim podstawieniu $x = 0$ otrzymujemy symbol nieoznaczony $\frac{0}{0}$. Daje nam to prawo do zastosowania reguły de l'Hospitala. Różniczkujemy zatem niezależnie licznik oraz mianownik:

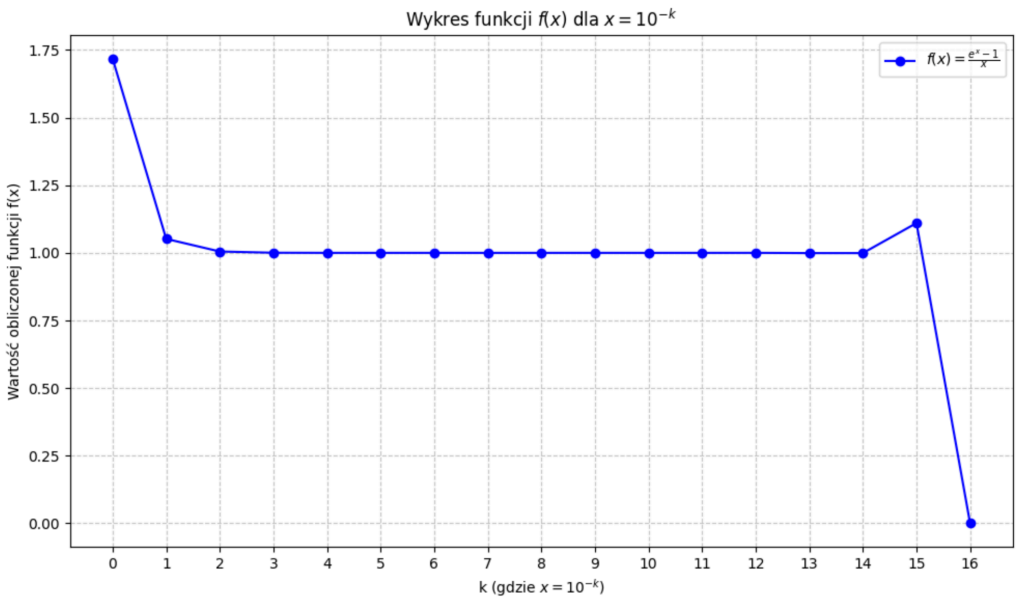
- Pochodna licznika: $(e^x - 1)' = e^x$
- Pochodna mianownika: $(x)' = 1$

Zapisując granicę ilorazu pochodnych, otrzymujemy ostateczny wynik:

$$\lim_{x \to 0} \frac{e^x - 1}{x} = \lim_{x \to 0} \frac{e^x}{1} = \frac{e^0}{1} = \frac{1}{1} = 1 \tag{12}$$

Tym samym wykazano, że $\lim_{x \to 0} f(x) = 1$, co pokrywa się z wartością funkcji zdefiniowaną dla $x = 0$.

3.2.2. Wykresy

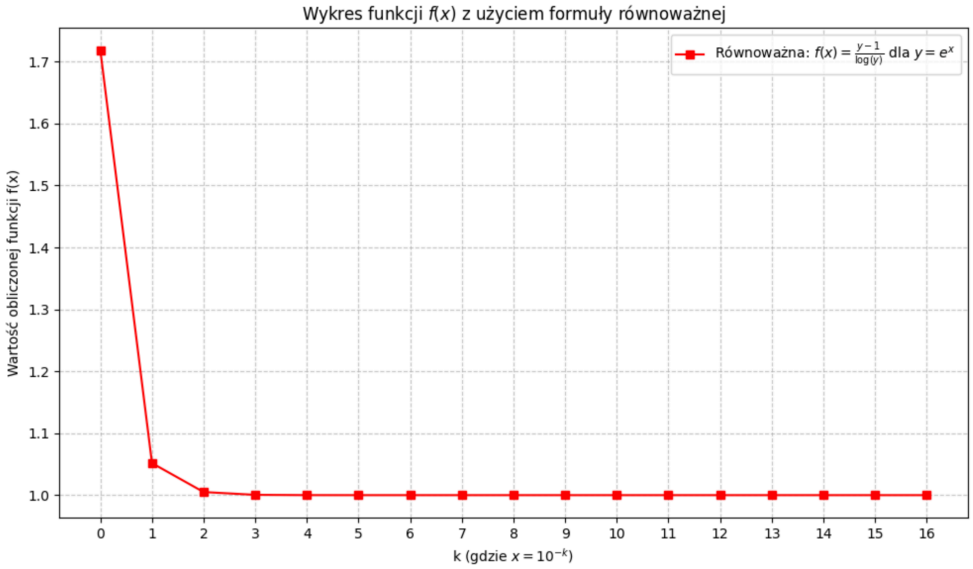


Jak widać na powyższym wykresie, dla $x < 10^{-15}$ błąd numeryczny zaczyna dominować, a dla $x = 10^{-16}$ wynik całkowicie spada do zera. Zjawisko to wynika z ograniczeń precyzji maszynowej i występowania tzw. błędu obcięcia przy odejmowaniu bardzo bliskich sobie wartości ($e^x \approx 1$).

Aby sprawdzić, czy problem ten można ominąć poprzez proste przekształcenie algebraiczne, zgodnie z poleceniem wprowadzamy matematycznie równoważną formułę. Stosujemy podstawienie $y = e^x$, co daje nam nową postać funkcji:

$$f(x) = \begin{cases} 1 & \text{dla } y = 1 \\ \frac{y-1}{\log(y)} & \text{dla } y \neq 1 \end{cases} \tag{13}$$

Kolejny wykres przedstawia zachowanie tej zmodyfikowanej formy. Celem tego kroku jest weryfikacja, czy taka zmiana postaci analitycznej pozwoli nam uniknąć problemu kasowania cyfr znaczących podczas obliczeń zmiennoprzecinkowych.



Jest to postać stabilniejsza numerycznie dla małych x.

Aby poprawić własności numeryczne funkcji dla małych wartości x, możemy skorzystać z rozwinięcia funkcji e^x w szereg Taylora (szereg Maclaurina wokół punktu $x_0 = 0$):

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + ... = \sum_{n=0}^{\infty} \frac{x^n}{n!} \tag{14}$$

Podstawiając to rozwinięcie do oryginalnego wzoru na funkcję $f(x)$, otrzymujemy:

$$f(x) = \frac{e^x - 1}{x} = \frac{\left(1 + x + \frac{x^2}{2} + \frac{x^3}{6} + ...\right) - 1}{x} \tag{15}$$

Po redukcji jedynek w liczniku wzór upraszcza się do postaci:

$$f(x) = \frac{x + \frac{x^2}{2} + \frac{x^3}{6} + ...}{x} \tag{16}$$

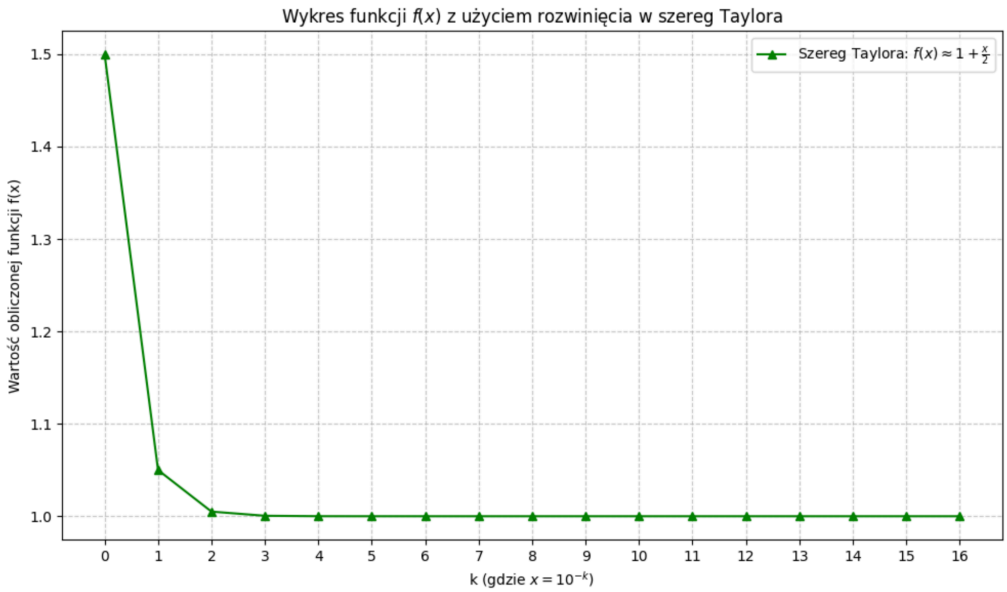
Wyciągając x przed nawias (pamiętając, że badamy $x \neq 0$), możemy skrócić ułamek:

$$f(x) = \frac{x\left(1 + \frac{x}{2} + \frac{x^2}{6} + ...\right)}{x} = 1 + \frac{x}{2} + \frac{x^2}{6} + ... \tag{17}$$

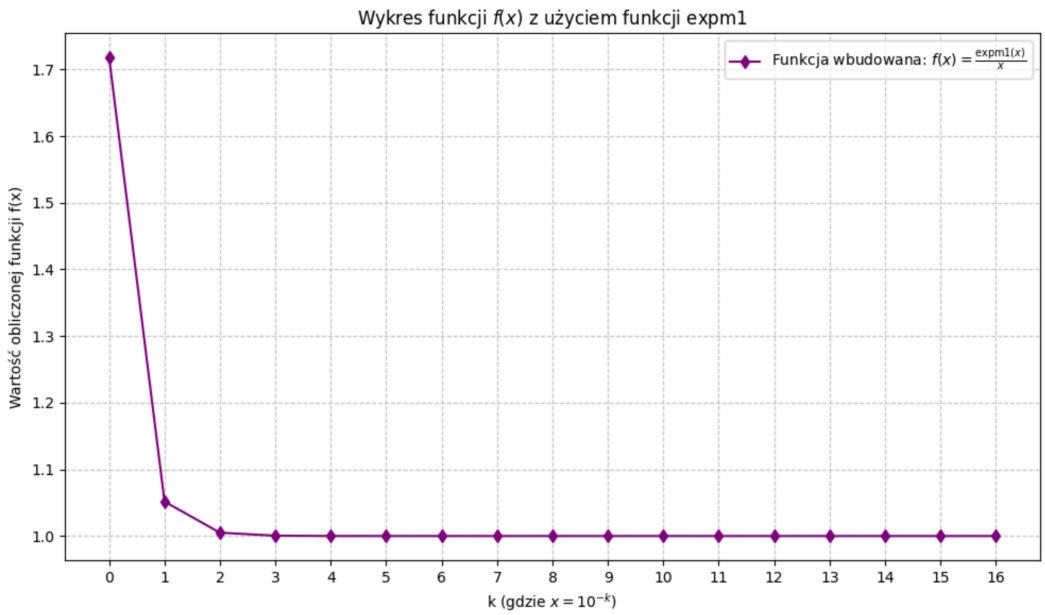
Dla wartości x dążących do zera, wyrażenia z wyższymi potęgami x stają się pomijalnie małe. Dlatego do analizy numerycznej dla bardzo małych x (gdzie tradycyjny wzór zawodzi) możemy z powodzeniem zastosować przybliżenie pierwszego rzędu:

$$f(x) \approx 1 + \frac{x}{2} \tag{18}$$

Taka postać analitycznie eliminuje operację odejmowania bliskich sobie liczb ($e^x \approx 1$ dla $x \approx 0$). Dzięki temu całkowicie omijamy zjawisko redukcji cyfr znaczących i związany z nim duży błąd względny, zyskując precyzyjny wynik pomimo ograniczeń arytmetyki zmiennoprzecinkowej.

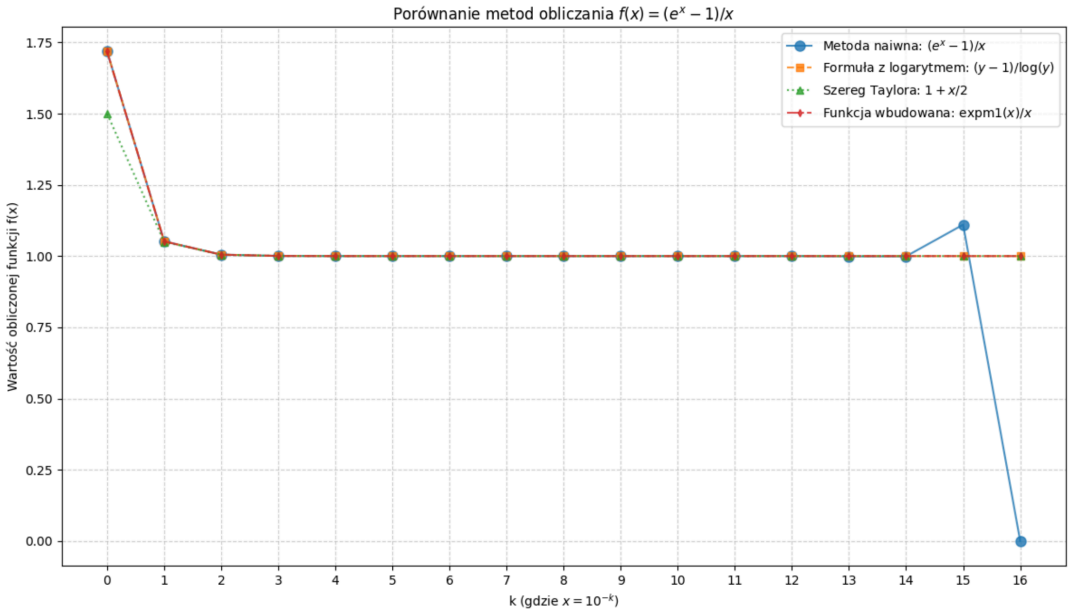


Ostatnim badanym sposobem jest wykorzystanie wbudowanej funkcji bibliotecznej `expm1(x)`. Funkcja ta jest zaimplementowana w sposób gwarantujący wysoką precyzję obliczania wyrażenia $e^x - 1$ dla wartości x bliskich zeru.



Jak ukazuje powyższy wykres, wynik dla funkcji `expm1` pozostaje stabilny i zbiega do wartości 1 dla każdego badanego k.

Na poniższym wykresie można zobaczyć porównanie wszystkich metod obliczania badanej funkcji.



3.3. Wnioski

Analiza numeryczna funkcji $f(x) = \frac{e^x - 1}{x}$ wykazuje, że stosowanie wzoru bezpośredniego dla x bliskich zeru prowadzi do katastrofalnej utraty cyfr znaczących z powodu odejmowania bardzo bliskich sobie wartości. Przy skrajnie małych argumentach ($x = 10^{-16}$), błąd reprezentacji zmiennoprzecinkowej sprawia, że e^x staje się maszynowo równe 1, co skutkuje błędnym wynikiem 0. Wykorzystanie matematycznie równoważnej formuły z logarytmem oraz jawna obsługa warunku brzegowego pozwalają na odzyskanie stabilności numerycznej. Najdokładniejsze i najbardziej odporne na błędy zaokrągleń okazują się jednak metody eliminujące kłopotliwe odejmowanie na etapie analitycznym: zastosowanie przybliżenia szeregiem Taylora ($1 + \frac{x}{2}$) oraz użycie zoptymalizowanej funkcji bibliotecznej `expm1`.

4. Zadanie 4

4.1. Treść zadania

Napisz program obliczający sumę n liczb zmiennoprzecinkowych pojedynczej precyzji, losowo rozłożonych w przedziale $[0, 1]$ wg rozkładu jednostajnego. Porównaj następujące metody:

1. Sumowanie w kolejności generowania — akumulator **podwójnej** precyzji.
2. Sumowanie w kolejności generowania — akumulator **pojedynczej** precyzji.
3. Algorytm Kahana (kompensacja błędu zaokrąglenia) — akumulator pojedynczej precyzji.
4. Sumowanie w kolejności **rosnącej** (od najmniejszych wartości bezwzględnych).
5. Sumowanie w kolejności **malejącej** (od największych wartości bezwzględnych).

Narysuj wykres błędu względnego w zależności od $n = 10^k$, $k = 4, ..., 8$. Jako wartość prawdziwą przyjmij `math.fsum(x)`.

4.2. Wstęp

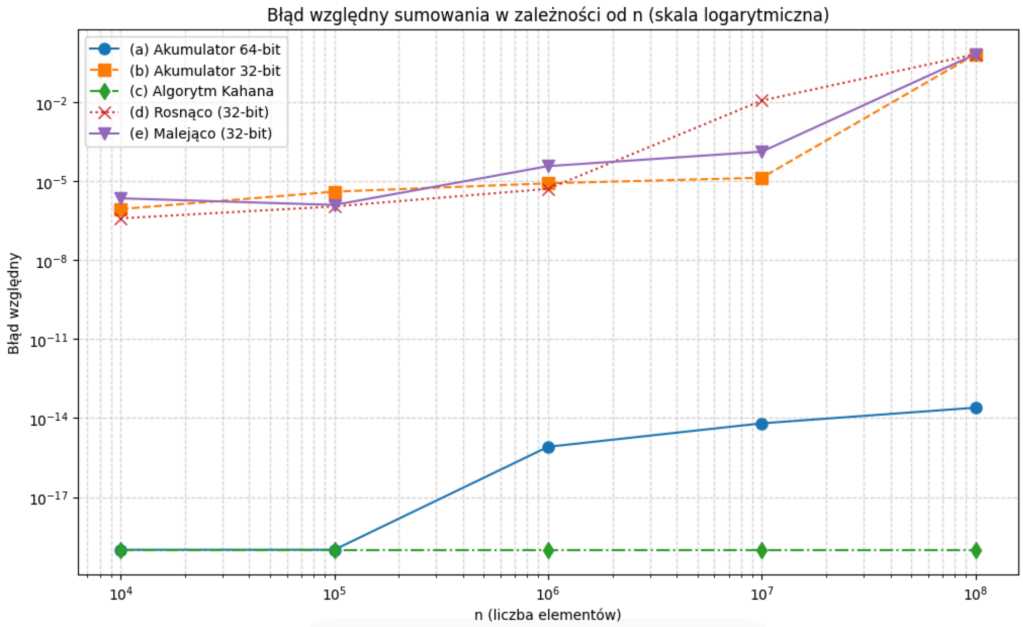
Celem zadania jest porównanie precyzji sumowania n liczb losowych z przedziału $[0, 1]$ dla $n \in \{10^4, 10^5, 10^6, 10^7, 10^8\}$ przy użyciu różnych strategii akumulacji i sortowania danych. Jako wartość referencyjną przyjęto wynik funkcji `math.fsum`.

4.2.1. Mechanizm akumulacji błędów i implementacja

Głównym źródłem błędów w standardowych metodach 32-bitowych jest zjawisko zaokrąglania młodszych bitów mniejszego składnika przy dodawaniu go do znacznie większej sumy bieżącej. Aby zminimalizować ten efekt, obok prostych metod sortowania, zastosowano **algorytm Kahana**, który wprowadza zmienną poprawkową `err` przechowującą niedokładność powstałą w poprzednim kroku:

```
def kahan_sum(xs):
    s = np.float32(0.0)
    err = np.float32(0.0)
    for x in xs:
        y = np.float32(x - err)
        temp = np.float32(s + y)
        err = np.float32(np.float32(temp - s) - y)
        s = temp
    return s
```

4.3. Wykres



Powyższy wykres przedstawia zależność błędu względnego od liczby sumowanych elementów n w skali logarytmicznej. Na podstawie przebiegu krzywych można sformułować następujące obserwacje:

1. **Metody standardowe 32-bitowe (b, d, e):** Wykazują najwyższy poziom błędu, który wyraźnie narasta wraz ze zwiększaniem liczby operacji n. Wynika to z faktu, że przy każdej iteracji suma bieżąca staje się coraz większa w stosunku do kolejnych składników, co prowadzi do utraty cyfr znaczących.
2. **Wpływ sortowania (d, e):** Sumowanie w porządku rosnącym (linia d) generuje mniejsze błędy niż w porządku malejącym (linia e). Dzieje się tak, ponieważ dodawanie do siebie liczb o zbliżonych rzędach wielkości minimalizuje błąd bezwzględny pojedynczej operacji.
3. **Akumulator 64-bitowy (a):** Dzięki znacznie większej precyzji (16 cyfr dziesiętnych), błąd pozostaje na niskim poziomie (ok. 10^{-16} do 10^{-14}), co odpowiada maszynowej precyzji formatu Double.

4. **Algorytm Kahana (c):** Reprezentowany przez najniższą, stabilną linię na wykresie. Dzięki mechanizmowi kompensacji błędu, algorytm ten utrzymuje niemal zerowy błąd względny, co czyni go najbardziej odpornym na wzrost parametru n.

4.4. Wnioski

Ćwiczenie dowiodło, że podczas sumowania dużej liczby składników w arytmetyce zmiennoprzecinkowej, błąd względny rośnie proporcjonalnie do liczby operacji n, o ile nie zostanie zastosowany mechanizm kompensacji. Standardowe metody 32-bitowe wykazują najniższą stabilność, ponieważ przy dodawaniu małych wartości do dużej sumy bieżącej następuje nieuchronne odcięcie młodszych bitów mantysy składnika. Sortowanie danych (metody d i e) jedynie częściowo łagodzi ten problem. Sumowanie rosnące jest korzystniejsze, gdyż dłużej utrzymuje zbliżone rzędy wielkości dodawanych liczb, spowalniając narastanie błędu bezwzględnego. Jednak w miarę powiększania się sumy błąd zaczyna nawet przewyższać zwykły akumulator 32-bitowy. Wynika to z faktu, że przy rosnącej sumie bieżącej, posortowane małe składniki na pewnym etapie przestają w ogóle wpływać na wynik (ich wartość jest mniejsza niż epsilon maszynowy aktualnej sumy), co prowadzi do pominięcia ogromnej liczby danych w końcowej fazie algorytmu.

Najskuteczniejszym rozwiązaniem okazał się algorytm Kahana, który dzięki wprowadzeniu zmiennej korygującej pozwala na „odzyskanie” błędu zaokrąglenia i uwzględnienie go w kolejnej iteracji. Wyniki eksperymentu jednoznacznie pokazują, że poprawność numeryczna zależy w większym stopniu od doboru stabilnego algorytmu niż od samej precyzji użytego typu danych.